Embedded Real Time Systems

Assignment 2 – Building SoPC Designs on the ZYBO FPGA Platform

Student Name	Student ID
Firas Harbo Saleh	202109174

11 October 2026

Introduction

This journal documents the mandatory exercises 3, 4, 5, and 7 from Assignment 2 in Embedded Real Time Systems. The work progresses from bare-metal control on the Zynq processing system to custom acceleration in programmable logic. The implementation includes a UART command interpreter, GPIO and timer control, software and hardware matrix multiplication, and a fixed-point neural network implemented with High Level Synthesis.

The software matrix implementation and the neural-network C model were verified with self-checking tests. The neural-network design was synthesized for the XC7Z010, passed VHDL C/RTL co-simulation, was implemented in Vivado 2026.1, and was executed on the ZYBO as a native Cortex-A9 application built by Vitis 2026.1. All ten accelerator outputs matched the software reference. The application screenshots are direct captures from the earlier native session on 10 October, retained as historical views of the tools. The updated measurements use raw UART and build logs from 11 October, not reconstructed screenshots.

The course specifies 2017.2; acceptance of 2026.1 is not yet established. The original ANN testbench and the documented adapted fixed-point reference also differ, as explained in the verification section. These boundaries prevent an unqualified claim that every written acceptance condition is complete.

Table 1: Implementation and verification overview

Exercise	**Implementation**	**Verification state**
3	UART, custom LED IP, timer, OCM	Native build/UART passed; user observed full LED counter
4	4×4 software matrix multiplication	Host and native ZYBO result passed
5	AXI matrix accelerator integration	Three native software/hardware comparisons passed
7	Fixed-point ANN in HLS and Vitis	Adapted-model verification; original-reference acceptance remains open

Contents

1 Exercise 3: UART Console Application **3**

1.1 Task . . . 3

1.2 Design . . . 3

1.3 Native Build and Physical Result . . . 4

2 Exercise 4: Software Matrix Multiplication **4**

2.1 Task and Data Representation . . . 4

2.2 Implementation . . . 4

2.3 Verification . . . 5

3 Exercise 5: Hardware Matrix Accelerator **5**

3.1 Hardware and Software Partition . . . 5

3.2 Benchmark Method . . . 6

3.3 Discussion . . . 6

4 Exercise 7: HLS Neural Network Accelerator **6**

4.1 Network Structure . . . 6

4.2 Fixed-Point Implementation . . . 7

4.3 Testbench . . . 7

4.4 Baseline and Pipelined Synthesis . . . 8

4.5 C/RTL Co-Simulation and Export . . . 8

4.6 Vitis Integration and Target Benchmark . . . 11

4.7 Fresh Native UART Transcript . . . 13

5 Conclusion **13**

A Source File Map **14**

1 Exercise 3: UART Console Application

1.1 Task

The application must implement a command language controlled through USB-UART. Command `1` reads switches SW0–SW3 and displays the four-bit value on LED0–LED3. Command `2` presents a binary counter on the LEDs and advances once per second.

1.2 Design

The switches use AXI GPIO configured as input. The supplied custom LED IP is controlled by a write to its register 0; it is not an AXI GPIO device. The generated address map supplies each peripheral address. Under the modern SDT driver flow GPIO initialization uses its base address; a separate legacy path retains SDK device-ID lookup. The Cortex A9 private SCU timer supplies the delay. The timer runs at half of the 650 MHz processor clock, so a one-second interval is represented by 325,000,000 timer ticks. A line-oriented parser accepts complete commands and reports invalid input rather than silently ignoring it.

Listing 1: UART command dispatch

```
if (strcmp(command, "1") == 0) {
    copySwitches();
} else if (strcmp(command, "2") == 0) {
    runCounter();
} else if (strcmp(command, "help") == 0) {
    showHelp();
} else {
    xil_printf("Unknown command '%s'.\r\n", command);
}
```

The switch command masks the read value to four bits before writing it to the custom LED register. This prevents unrelated bits from affecting the result.

Listing 2: Switch-to-LED transfer

```
const u32 value = XGpio_DiscreteRead(&switches, GPIO_CHANNEL)
                  & LED_MASK;
Xil_Out32(ERTS_LED_BASEADDR, value);
xil_printf("Switch value: 0x%x\r\n", value);
```

The counter writes all sixteen possible four-bit patterns. Polling the timer is appropriate here because the exercise intentionally demonstrates basic bare-metal control and no other task must execute concurrently.

Listing 3: One-second binary LED counter

```
// Timer is started once with auto-reload at 325,000,000 ticks.
for (value = 0U; value <= LED_MASK; ++value) {
    setLeds(value);
    xil_printf("Counter: %d\r\n", value);
    if (value != LED_MASK) {
        while (!XScuTimer_IsExpired(&timer.instance)) {}
        XScuTimer_ClearInterruptStatus(&timer.instance);
    }
}
// Stop the timer and disable auto-reload after the final value.
```

1.3 Native Build and Physical Result

The application was built with Vitis 2026.1 at `-O2` on the supplied Ex5 platform reconstructed for 2026.1. It retains the Ex3 processor, switches and custom LEDs while also providing the matrix IP used by Exercise 5. The selected linker script puts the heap at `0x1000-0x2FFF` and all stack modes at `0x3000-0x67FF`; inspection of the actual ARM ELF verifies these OCM ranges and retention of the SDT driver configuration section.

Real UART returned switch value `0xF`, every counter value 0–15, and the correct matrix result for the added `mult` command. Host UART arrival intervals on the first run were approximately 0.992–1.008 seconds; USB scheduling introduces jitter. During the requested rerun, the user confirmed: "Yes, the full count looked correct" in response to observing all four LEDs from 0000 to 1111 at about one step per second. This is physical user observation, not a camera recording. A separate switch-by-switch mapping test remains pending.

2 Exercise 4: Software Matrix Multiplication

2.1 Task and Data Representation

Three global arrays named `aInst`, `bTInst`, and `pInst` store matrix A, the transpose of matrix B, and the product P. Each row uses a union named `vectorType`; `VectorArray` is an array of four rows. The component view is convenient for software, while the packed 32-bit view is reused by the hardware accelerator in Exercise 5.

Listing 4: Packed matrix row

```
typedef union {
    unsigned char comp[MSIZE];
    unsigned int vect;
} vectorType;
typedef vectorType VectorArray[MSIZE];

void setInputMatrices(VectorArray A, VectorArray B);
void displayMatrix(VectorArray input);
void multiMatrixSoft(VectorArray A, VectorArray B, VectorArray P);
void multiMatrixHard(VectorArray A, VectorArray B, VectorArray P);
```

Table 2: Input matrices and expected product

	A				C (stored B^T)				$P = AC^T$			
0	1	2	3	4	1	2	3	4	30	30	30	30
1	5	6	7	8	1	2	3	4	70	70	70	70
2	9	10	11	12	1	2	3	4	110	110	110	110
3	13	14	15	16	1	2	3	4	150	150	150	150

2.2 Implementation

The stored layout follows the literal transpose printed on assignment p. 6. Writing C for the stored array avoids ambiguity: the operation is $P_{ij} = \sum_k A_{ik} C_{jk} \pmod{256}$. The previous constant-row input layout remains a separate regression test, not the submitted fixture.

For every output element, the algorithm calculates the dot product of one row from A and one row from the stored transpose of B. The three nested loops perform $4 \times 4 \times 4 = 64$ byte multiplications.

Listing 5: Software matrix multiplication

```
for (row = 0; row < MSIZE; ++row) {
    for (col = 0; col < MSIZE; ++col) {
        unsigned int sum = 0U;
        for (k = 0; k < MSIZE; ++k) {
            sum += (unsigned int)A[row].comp[k] * B[col].comp[k];
        }
        P[row].comp[col] = (unsigned char)sum;
    }
}
```

The command `mult` initializes the matrices, starts the SCU timer, calls the software multiplication, stops the timer, and prints all three matrices and the elapsed tick count to USB-UART.

2.3 Verification

A host-side unit test compares all sixteen calculated elements with the expected product in Table 2. The expanded suite covers the course transpose, the previous fixture, non-symmetric rows and modulo-256 overflow; all pass. The standalone Ex4 application returned the same matrix on the ZYBO and measured 260 SCU timer ticks, or 0.800 μs at 325 MHz. This is one observed `-O2` run, not a statistical CPU benchmark.

The reproducible host command is `make -C Assignment_2 test`; it exits successfully after printing `matrix test:  PASS`.

3 Exercise 5: Hardware Matrix Accelerator

3.1 Hardware and Software Partition

The custom matrix IP replaces the inner software loop. The processor writes one packed row from A to AXI register 0 and one packed row from B^T to AXI register 1. Programmable logic performs four parallel eight-bit products, adds the partial results, and exposes the dot product in register 2.

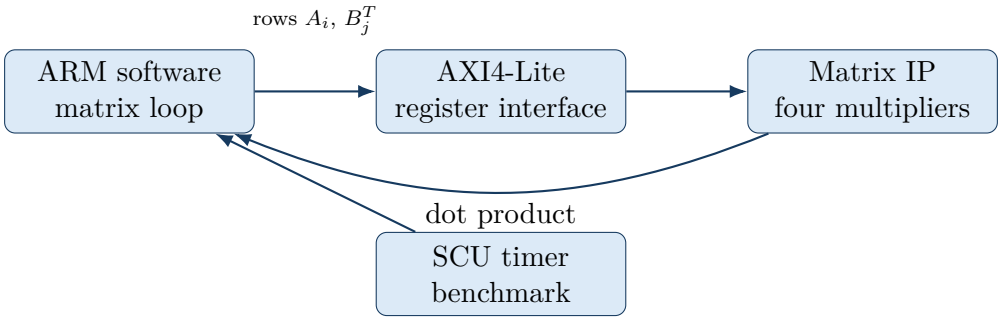


Figure 1: Matrix accelerator data path

Listing 6: Memory-mapped accelerator access

```
Xil_Out32(BASE + REG0_OFFSET, a[row].vect);
Xil_Out32(BASE + REG1_OFFSET, bT[col].vect);
product[row].comp[col] =
    (uint8_t)Xil_In32(BASE + REG2_OFFSET);
```

3.2 Benchmark Method

The application measures both implementations with identical inputs. The software interval contains all 64 processor multiplications. The hardware interval contains sixteen accelerator calls, including two AXI writes and one AXI read for each output. Finally, all sixteen hardware outputs are compared with the software reference.

Table 3: Exercise 5 native ZYBO measurements, application `-O2`

Run	**Software ticks / time**	**Hardware ticks / time**
First	264 / 0.812 μs	3,190 / 9.815 μs
Second	197 / 0.606 μs	3,116 / 9.588 μs
Third	192 / 0.591 μs	3,122 / 9.606 μs

3.3 Discussion

The exercise exposes the central tradeoff in acceleration. Parallel arithmetic reduces computation latency, but memory-mapped communication adds overhead. For this small matrix, AXI transfers may dominate. Larger workloads, batching, streaming, or local buffering would amortize the transfer cost more effectively. All three runs returned `Verification: PASS`; the capture also checked both output matrices against the independently known literal-transpose result. The 650 MHz ARM executes this small optimized loop faster than sixteen sets of AXI register transfers to the 100 MHz PL. Timer overhead is included and is not subtracted; first-run cache effects and limited sample count qualify the figures.

The Ex345 implementation met timing with WNS +1.216 ns, WHS +0.046 ns, zero negative slack and zero final DRC violations. Original board DDR preset skew warnings were retained during BD validation; the generated native apps subsequently ran successfully on this board.

4 Exercise 7: HLS Neural Network Accelerator

4.1 Network Structure

The accelerator implements a fully connected network with 196 signed eight-bit inputs, 32 hidden neurons, and 10 outputs. Input and output weights are signed 16-bit Q15 values. Each dot-product accumulator is shifted right by 15 bits to restore the numeric scale. Negative hidden-layer values are replaced by zero with a ReLU activation function.

Table 4: Neural-network parameters

Parameter	Value	Meaning
`INPUT_LEN`	196	Downsampled image input values
`NEURONS_LEN`	32	Hidden-layer neurons
`OUTPUT_LEN`	10	Output classes
`BITS`	15	Fractional weight bits
Weight arrays	AXI4-Lite	Processor-controlled memories
Switch and LED ports	4 bits	Direct pass-through demonstration

4.2 Fixed-Point Implementation

Weights are stored neuron-major. Each hidden neuron consumes a contiguous block of 196 input weights, and each output consumes 32 output weights. The indexing is important: using the number of neurons rather than the input length as the first stride would mix weights between neurons.

Listing 7: Hidden and output layer calculations

```
accumulator += (ap_int<8>)input[j]
               * (ap_int<16>)weightsI[i * INPUT_LEN + j];
neuron = (int)(accumulator >> BITS);
neurons[i] = neuron > 0 ? neuron : 0;

accumulator += (ap_int<32>)neurons[j]
               * (ap_int<16>)weightsO[i * NEURONS_LEN + j];
output[i] = (int)(accumulator >> BITS);
```

The arrays are mapped to AXI4-Lite interfaces. The switches and LEDs use direct ports so that the synthesized IP can demonstrate a visible hardware connection.

Listing 8: HLS interface directives

```
#pragma HLS INTERFACE ap_none port=inSwitch
#pragma HLS INTERFACE ap_none port=outLeds
#pragma HLS INTERFACE s_axilite port=input bundle=BUS
#pragma HLS INTERFACE s_axilite port=weightsI bundle=BUS
#pragma HLS INTERFACE s_axilite port=weightsO bundle=BUS
#pragma HLS INTERFACE s_axilite port=output bundle=BUS
#pragma HLS INTERFACE s_axilite port=return bundle=BUS
```

4.3 Testbench

An independent 64-bit fixed-point reference traverses each weight array in sequence and implements floor division explicitly for negative Q15 values. Five cases cover the adapted course vector, bounded signed values, zero input after a nonzero inference, negative hidden activations, and signed extremes. Every case checks all outputs and the switch/LED port value. An additional 8/3/4-dimension configuration is checked on the host, not on the FPGA.

The original course testbench is preserved unchanged under `hls/provided_course/`. It uses overlapping weight-row strides and a floating-point reference; all ten comparisons fail against the

contiguous fixed-point model. The adapted course test retains the supplied data generation but corrects those reference assumptions. This documented deviation still requires course acceptance; passing the adapted test is not passing the untouched original.

The C-simulation transcript is stored with the HLS project and ends with `neural network test: PASS`. The independent reference calculation makes this a functional check rather than a comparison against copied accelerator code.

4.4 Baseline and Pipelined Synthesis

Two solutions were synthesized for `xc7z010clg400-1` with a 10 ns target clock. The baseline disables automatic loop pipelining. The optimized solution applies initiation interval 1 directives to both inner dot-product loops.

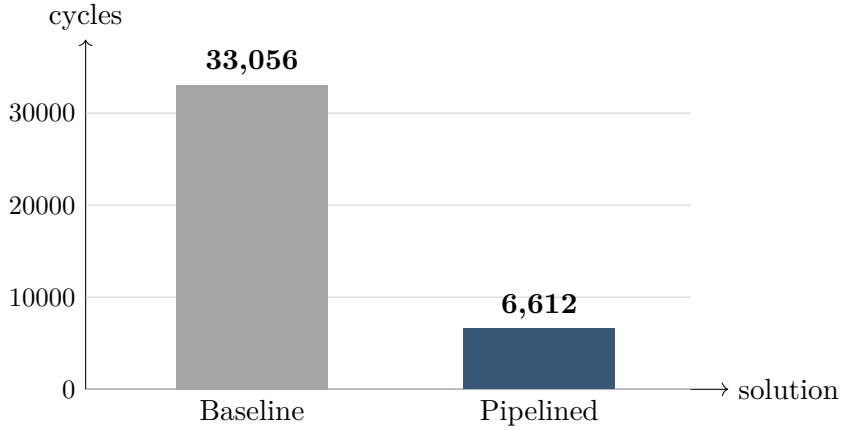


Figure 2: Estimated neural-network latency

Table 5: HLS synthesis comparison

Solution	Latency	BRAM 18K	DSP	FF	LUT
Baseline	33,056	10	2	638	716
Pipelined, II=1	6,612	10	3	1,112	1,126

Pipelining reduces estimated latency by approximately five times. The optimized solution uses one additional DSP, 474 additional flip-flops, and 410 additional lookup tables; BRAM usage is unchanged. The tool estimated a maximum frequency of approximately 194 MHz for the baseline and 143 MHz for the pipelined solution, both above the 100 MHz target constraint.

4.5 C/RTL Co-Simulation and Export

The optimized VHDL solution passed C/RTL co-simulation for all five test cases, including the adapted course vector. Reported latency was 6,610 / 6,612 / 6,613 cycles (minimum / average / maximum). The testbench transaction intervals include AXI stimulus/loading gaps and are not the II=1 inner-loop initiation interval. Post-checking confirmed every output and LED-port result. The core was exported as VHDL IP version 1.0.1 as required by the assignment.

The supplied Exercise 7 Vivado project was migrated from its legacy format to Vivado 2026.1. The regenerated HLS IP was connected through two AXI4-Lite interfaces, the renamed switch

and LED ports were repaired, and explicit ZYBO pin constraints were applied. Implementation completed without routing errors and met timing with 3.462 ns worst slack and zero total negative slack.

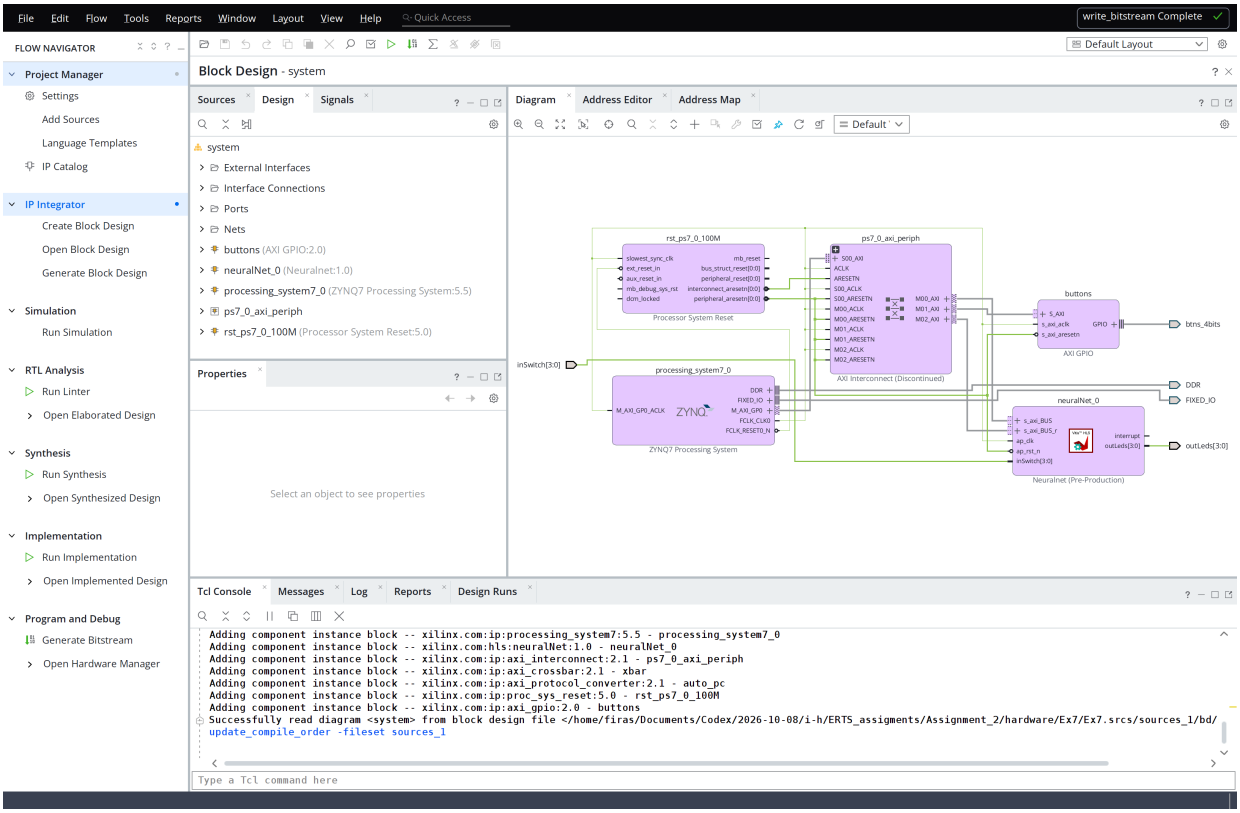


Figure 3: Historical native capture (10 October): Exercise 7 block design in Vivado 2026.1

The implemented design uses 959 LUTs, 1,097 flip-flops, six BRAM tiles, and three DSP slices. Figure 4 shows the completed native implementation run and its timing and utilization summaries.

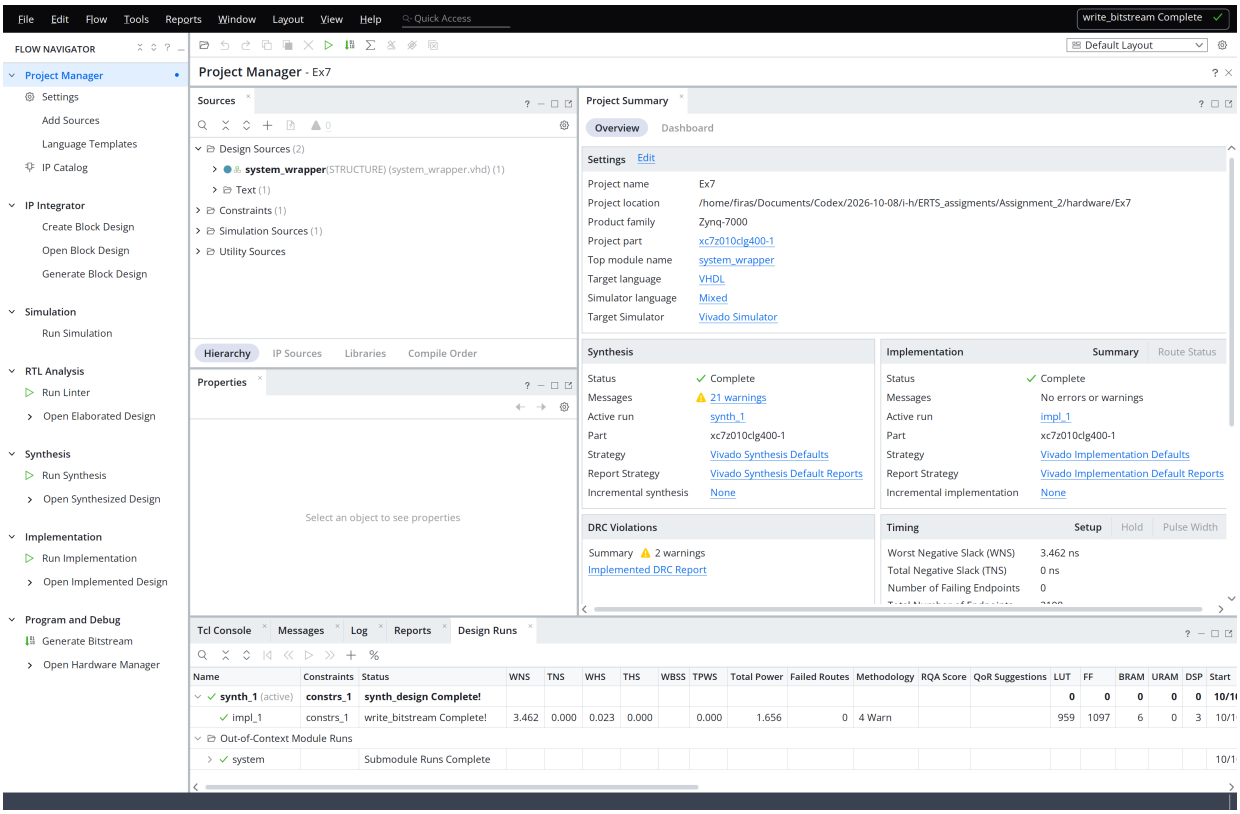


Figure 4: Historical native capture (10 October): completed Vivado project summary

The generated bitstream was programmed through the Digilent JTAG target. A Hardware Manager refresh returned DONE status 1 for `xc7z010_1`.

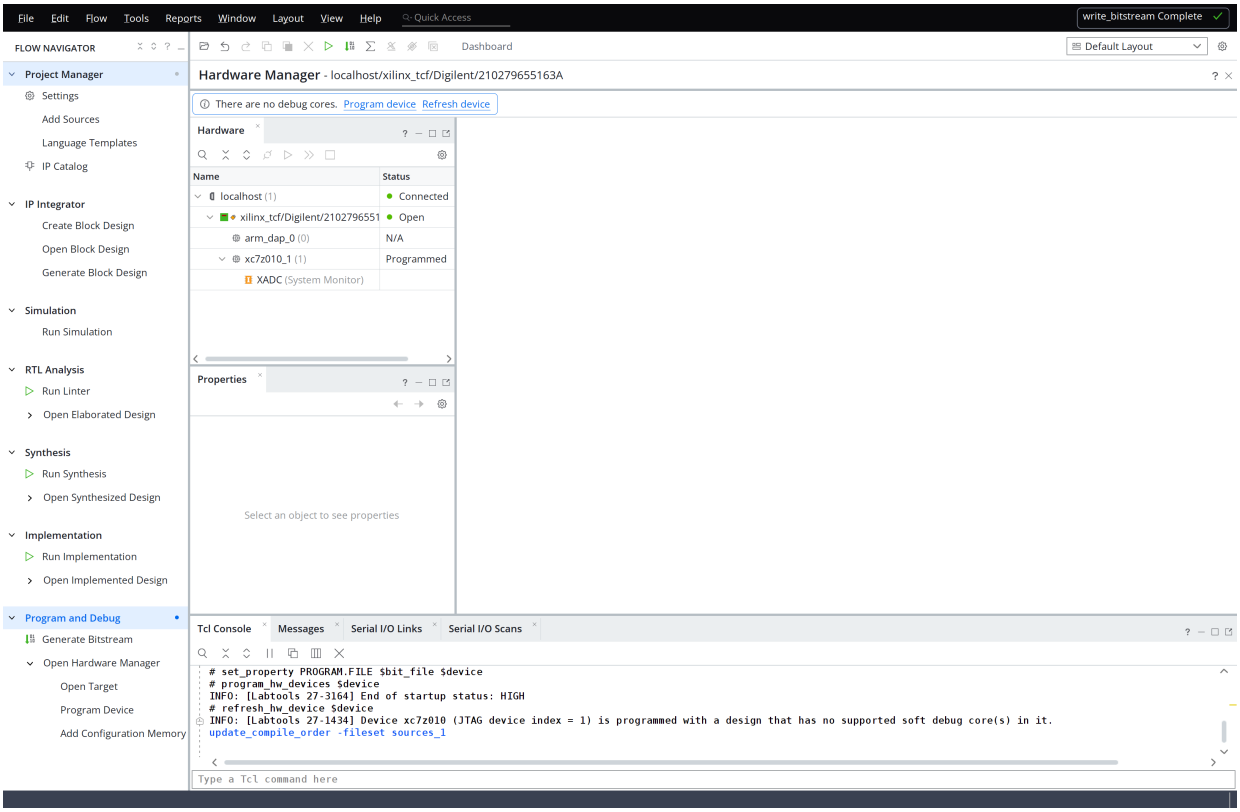


Figure 5: Historical native capture (10 October): Hardware Manager with ZYBO programmed

4.6 Vitis Integration and Target Benchmark

The Vitis application evaluates the network in software, transfers the input and both weight arrays through AXI, starts the accelerator, waits for completion, and reads ten outputs. CPU timing is separate; one continuous hardware timer measures the full transaction, with snapshots for loading and start-to-done. The native compiler command is verified as `-O2 -g3`.

Table 6: Exercise 7: ten subsequent verified trials at `-O2`

Measurement	**Min ticks**	**Mean ticks**	**Max ticks**	**Mean time**
CPU compute	13,844	14,373	14,757	44.2 μs
HW data loading	228,396	228,451	228,567	702.9 μs
HW start-to-done	21,623	21,632	21,636	66.6 μs
HW FULL transaction	253,315	253,386	253,559	779.6 μs

The 325 MHz SCU timer reports integer-mean ticks. The separate first-run CPU/load/run/FULL values were 14,789 / 233,803 / 21,648 / 259,189 ticks. FULL includes packing, all input/weight writes, readiness, execution, output readback and unpacking. It excludes platform setup, creation of test data, verification and UART output. Timer/snapshot overhead is included.

With optimization enabled, the ARM reference is approximately 1.51× faster than accelerator computation alone; the full hardware transaction is 17.63× slower than CPU compute. This supersedes the earlier 8.61× claim against an unoptimized `-O0` reference. Loading dominates the hardware path, and the 100 MHz core is not faster than the 650 MHz optimized ARM loop for this vector. The 5× HLS baseline-to-pipeline improvement is a different, still valid comparison.

All ten outputs matched on the initial and ten repeated transactions using the same custom signed vector and enabled caches. This is not a broad performance characterization or board execution of every HLS test vector. The fresh compiler/UART evidence is retained under `outputs/fix_evidence/`. Figure 6 is the earlier native workspace capture, not proof of the new compiler flags.

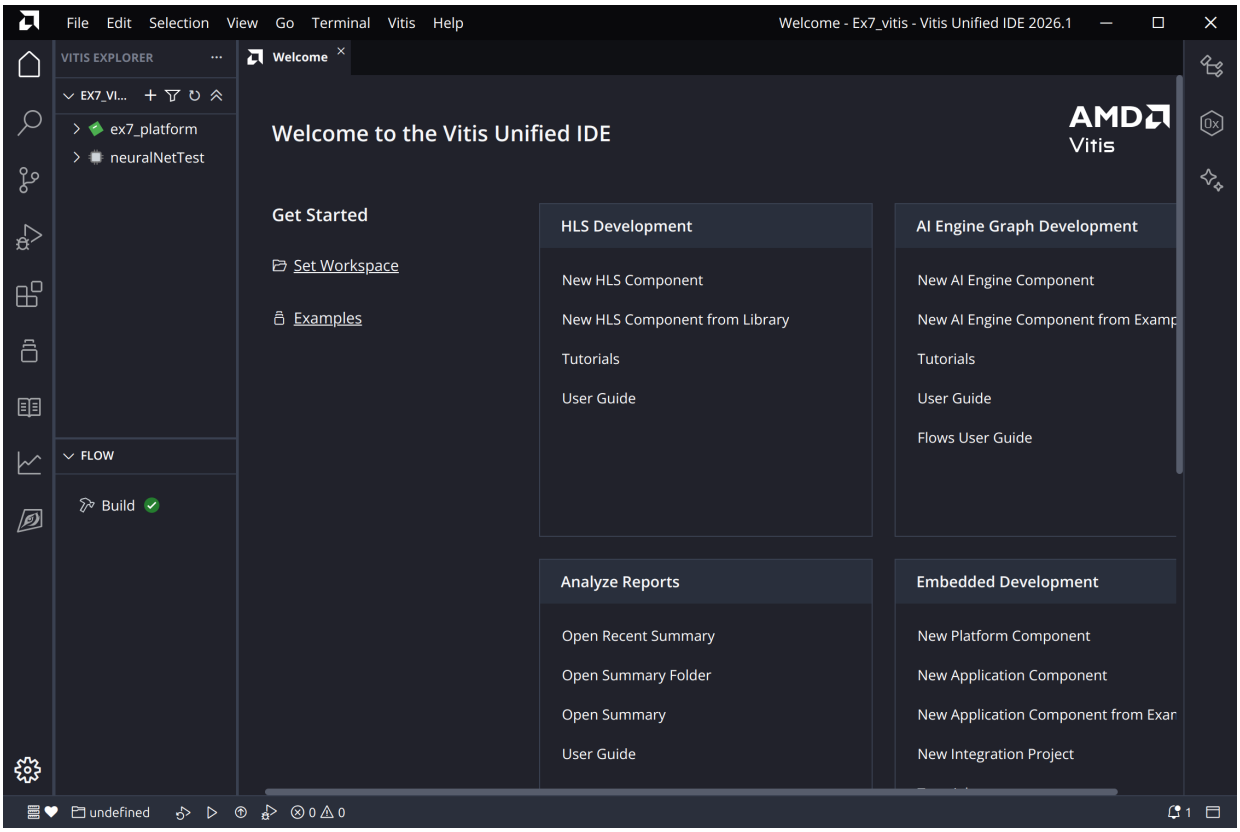


Figure 6: Historical native capture (10 October): Vitis 2026.1 application build

The fresh ELF was downloaded with XSDB after programming the rebuilt bitstream and initializing the processing system. Figure 7 preserves the earlier UART screenshot; its timing values are historical and are superseded by the fresh measurements above and raw transcript below.

```
ERTS Assignment 2 - ZYBO UART (115200 baud)


ERTS Assignment 2 - Exercise 7
output[0] SW=-4 HW=-4 PASS
output[1] SW=3 HW=3 PASS
output[2] SW=-2 HW=-2 PASS
output[3] SW=1 HW=1 PASS
output[4] SW=-4 HW=-4 PASS
output[5] SW=2 HW=2 PASS
output[6] SW=-1 HW=-1 PASS
output[7] SW=0 HW=0 PASS
output[8] SW=-3 HW=-3 PASS
output[9] SW=0 HW=0 PASS
CPU compute:     186751 ticks
HW data loading: 233860 ticks
HW compute:      21697 ticks
Overall verification: PASS
```

Figure 7: Historical native capture (10 October): earlier ZYBO run, before the `-O2` benchmark revision

4.7 Fresh Native UART Transcript

The following is a typeset copy of the actual captured serial bytes from the 11 October post-fix run, not a screenshot or reconstructed terminal image. Bitstream/ELF hashes are retained beside the log in `metadata.json`.

Listing 9: Actual post-fix ZYBO UART output

```
ERTS Assignment 2 - Exercise 7
Dimensions: input=196 hidden=32 output=10 Q bits=15
output[0] SW=-4 HW=-4 PASS
output[1] SW=3 HW=3 PASS
output[2] SW=-2 HW=-2 PASS
output[3] SW=1 HW=1 PASS
output[4] SW=-4 HW=-4 PASS
output[5] SW=2 HW=2 PASS
output[6] SW=-1 HW=-1 PASS
output[7] SW=0 HW=0 PASS
output[8] SW=-3 HW=-3 PASS
output[9] SW=0 HW=0 PASS
First CPU compute:         14789 ticks
First HW data loading:     233803 ticks
First HW compute:          21648 ticks
First HW FULL transaction: 259189 ticks
CPU reference (10 trials): min=13844 mean=14373 max=14757 ticks
HW data loading (10 trials): min=228396 mean=228451 max=228567 ticks
HW compute (10 trials): min=21623 mean=21632 max=21636 ticks
HW FULL transaction (10 trials): min=253315 mean=253386 max=253559 ticks
Overall verification: PASS
```

5 Conclusion

The assignment demonstrates a progression from processor-only control to hardware acceleration. Exercise 3 establishes UART, GPIO, and timer control. Exercise 4 provides a verified software reference. Exercise 5 moves the inner matrix operation into programmable logic and makes communication overhead explicit. Exercise 7 applies the same method to a fixed-point neural network and shows that targeted pipelining reduces latency substantially at moderate resource cost.

The post-review code fixes address peripheral selection, on-chip memory, course matrix interfaces, verification coverage and benchmark boundaries. Exercises 3–5 have native builds and real UART results, and the user confirmed the physical counter. The ANN uses a documented adapted reference rather than claiming that the original supplied test passes. Tool-version/reference acceptance and any unobserved switch/LED mapping remain explicit limitations.

Reproducibility

- Source branch: `assignment_2/Firas`
- Hardware implementation commit: `29f6373`
- Native target-validation commit: `cbec41c`
- Pre-fix review snapshot: `c38a216`; this revision includes local post-review fixes.
- Target part: `xc7z010clg400-1`

- HLS target clock: 10 ns
- Matrix test: `make -C Assignment_2 test`
- HLS flow: `vitis-run -mode hls -tcl run_hls.tcl`
- Vitis application: `vitis -s build_vitis_2026.py`
- Fresh build/UART logs and artifact hashes: `outputs/fix_evidence/`
- Three native apps and OCM proof: `Assignment_2/hardware/Ex345_vitis/`

A Source File Map

Exercise	Source location	Primary contents
3	`Assignment_2/exercise3_console/src/main.c`	UART, GPIO, counter
4	`Assignment_2/exercise4_matrix/src/main.c`	Software command and timing
4, 5	`Assignment_2/common/matrix.c`	Matrices and reference multiply
5	`Assignment_2/exercise5_matrix_ip/src/main.c`	AXI benchmark
7	`Assignment_2/exercise7_neural_net/hls/neuralNet.cpp`	Synthesizable ANN
7	`Assignment_2/exercise7_neural_net/hls/testbench.cpp`	Self-checking model
7	`Assignment_2/exercise7_neural_net/hls/run_hls.tcl`	HLS solutions
7	`Assignment_2/exercise7_neural_net/sdk/neuralNetTest.c`	Target benchmark